

CS361 Assignment 4

Tom Salaz

October 2025

Problem 1: Predecessor and Successor

Direct Proof by Contradiction of Claim: In a BST, if a node x has two children, then its successor has no left child and its predecessor has no right child.

Predecessor Definition: Node with the largest value in Left Subtree.

Successor Definition: Node with the smallest value in the Right Subtree.

Proof

(1) Since Tree x has a left child, the Predecessor P can be defined $P = \text{TreeMax}(x.\text{left})$, the rightmost node in the left sub tree.

For Contradiction, suppose that P has a right child, R . This implies that $P.\text{key} < R.\text{key}$. Now R is a node in $x.\text{left}$ and is the new maximum, contradicting the definition of P , thus P cannot have a right child.

(2) Since Tree x has a right child, the Successor S can be defined $S = \text{TreeMin}(x.\text{right})$, the leftmost node in the right sub tree.

For Contradiction, suppose that S has a left child, L . This implies that $L.\text{key} < S.\text{key}$. Now L is a node in $x.\text{right}$ and is the new minimum, contradicting the definition of S , thus S cannot have a left child.

Problem 2

Prove Algorithm has $O(n)$ runtime.

Given Algorithm

```
In_Order(x)
  x = Tree_Min(x.root)
  print x.key
  for i=1 to n-1:
    x = Tree_Succ(x)
    print x.key
```

Run Time Analysis

TreeMin Cost: Takes left edge until NULL, thus the runtime of this operation

is $O(h)$ where $h = \text{height}$.

TreeSucc Cost: There are 2 cases

1: Goes down to Right Child then Left until NULL

2. Goes up the tree until we find the first ancestor for the tree x in its left subtree.

Both operations are dependent on height so in the worst case, both have Run Time $O(h)$.

When analyzing for the Cost For the entire traversal (instead of per call), every edge in the BST is followed at most twice (once to go down for successor and once to go up for backtracking), thus the total edge traversals is $O(2n - 2) = O(n)$. Since $h \leq n$ the Runtime must follow the traversal case and not the per call case hence the Run Time is $O(n)$

Problem 3

Prove or Disprove: Given any Binary Tree, the number of leaf node is one more than the number of nodes with 2 children.

The Claim is True if the Tree is non-empty

BST properties and variables for proof:

Any tree with n nodes has $n - 1$ edges.

n_0 = number of leaves (0 children)

n_1 = number of nodes with 1 child

n_2 = number of nodes with 2 children

$n = n_0 + n_1 + n_2$

Proof: We need to define the number of children each node has, we can call this variable O , for how many nodes come out of a given node. O is defined as $O = \sum 0n_0 + 1n_1 + 2n_2$

We can reduce O to get $O = 1n_1 + 2n_2$ which is equivalent to $n - 1$. We can use this identity to relate n_0 to n_2

$$n_1 + 2n_2 = n - 1$$

$$n_1 + 2n_2 = (n_0 + n_1 + n_2) - 1$$

$$2n_2 = n_0 + n_2 - 1$$

$$n_2 = n_0 - 1$$

This is the same as

$$n_0 = n_2 + 1$$

Thus the number of leaves is one more than the number of nodes with 2 children